Metaexamples Boost Learning in Pre-Training

Kevin Lin Claude Code*

February 2026

Abstract

Treutlein et al. [1] show that an LLM fine-tuned on a class of examples is able to infer latent structures underlying those examples, even when the fine-tuning data does not *explicitly* provide any information about that latent structure. Their experiments use relatively simple latent structures (e.g., the bias of a coin). We extend this line of work to a more complex latent structure that the model struggles to learn from examples alone, and we ask: Can explicit natural language explanations of the latent structure, which we call *metaexamples*, help the model learn that structure?

Concretely, we create an artificial grammar with no semantic priors, and we perform continued pre-training of EleutherAI's Pythia 1.4B on mixtures of grammar examples and metaexamples that explain the grammar's rules. We find that adding a small amount of metaexamples to the training mix significantly improves the model's ability to produce valid grammar examples when training on top of early, middle, and late pre-training checkpoints. Surprisingly, the effect is strongest at the earliest checkpoint (∼2B tokens of pre-training), suggesting that the ability to relate descriptions to patterns emerges very early in pre-training.

1 Introduction

Treutlein et al. [1] demonstrate that LLMs can perform *inductive out-of-context reasoning* (OOCR)—inferring latent structures from disparate training examples and verbalizing them at test time. In one of their experiments, a model is fine-tuned on documents of the form

`from coins import CoinA; print(CoinA.flip())` ⟶ `Heads` or `Tails`

repeated across hundreds of examples with varying outcomes. The model never sees an explicit statement of the coin's bias. Yet when asked "What is the probability that CoinA lands heads?", it can give a reasonable estimate. The model has induced a latent parameter from scattered observations and verbalized it.

This is a striking and somewhat mysterious result. The model is not merely memorizing and recombining training data like a "stochastic parrot" [4]—it is also performing induction over the training data, extracting structure that was never explicitly stated. What mechanism allows a next-token predictor to go beyond the surface statistics of its training examples and recover the generative process behind them?

A clue is offered by "Textbooks Are All You Need" [2], which showed that training on textbook data dramatically improves sample efficiency—their 1.3B parameter model matched much larger

*Code available at `https://github.com/tractatus1889/hyperdata`

models on code generation. We observe that textbooks are notable in that they often contain two types of information side-by-side:

1. *Examples* of phenomena—worked problems, code snippets, instances (like the results of coinflips), and
2. *Metaexamples*—descriptions of the underlying structure or rules that govern those phenomena—definitions, explanations, theorems (like a coin's bias).

This suggests that textbooks and similar texts may help LLMs learn a bridge between examples and metaexamples. In this view, we may rephrase the result of [1] as: If a model has learned to bridge examples and metaexamples, then fine-tuning on examples alone will make the model implicitly infer the underlying structure which would be explained by metaexamples—at least for sufficiently simple latent structures.

In this paper, we extend the work of [1] to a setting where the latent structure is more complex—a novel grammar with multiple interacting constraints—and the model cannot fully learn it from examples alone. We ask: If we explicitly provide natural language descriptions of the latent structure (metaexamples) alongside examples, can the model integrate this knowledge to learn more effectively?

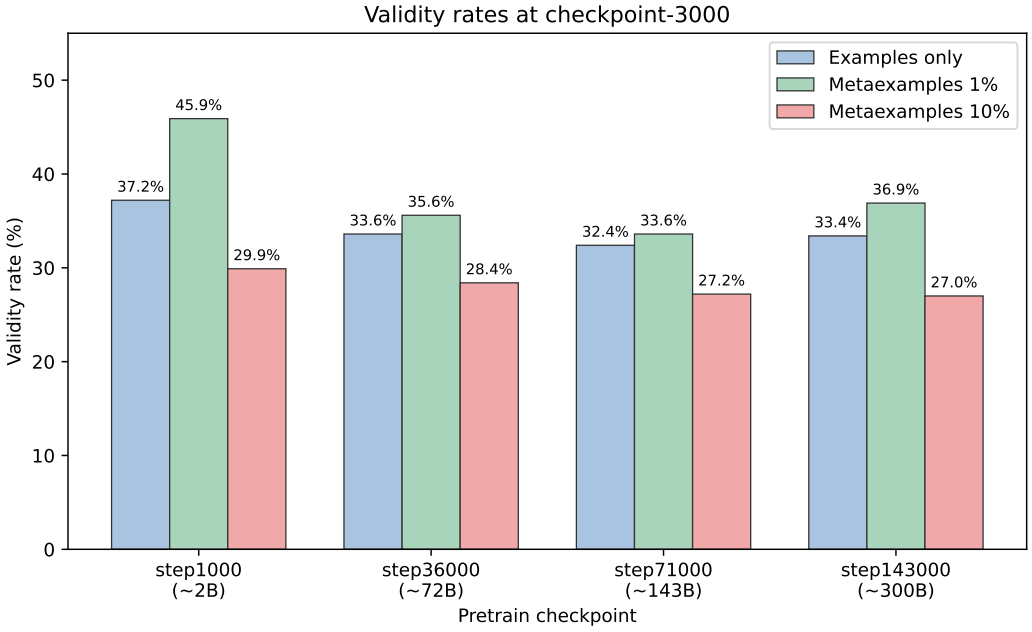

Figure 1: Validity rates at checkpoint-3000. Metaexamples 1% (green) outperforms examples-only (blue) at every pre-train checkpoint, with the largest gap at step1000. Metaexamples 10% (red) consistently underperforms.

To investigate this question, we create an artificial grammar, and we generate

1. *Examples* from this grammar, and
2. *Metaexamples* consisting of natural language descriptions of aspects of the grammar.

We perform continued pre-training of an open source pre-trained LLM on different mixtures of these examples and metaexamples. Note that unlike [1], who fine-tune a post-trained (instruction-tuned)

model, we train directly on pre-trained checkpoints—isolating the effect of pre-training alone without the confound of post-training. Our main result is that metaexamples boost the learning of examples, even for pre-trained (not post-trained) models. As shown in Figure 1, adding just 1% metaexamples to the training mix improves grammar validity at every level of pre-training maturity, with the strongest effect at the earliest checkpoint (step1000, ∼2B tokens of pre-training).

2 Grammar

We define a fictional grammar that we call Tivari consisting of nonsense tokens and arbitrary rules. We do this to minimize the possibility that the pre-trained model has any priors on the grammar. This way, the model's learning of the grammar must be done "from scratch".

The grammar is defined as follows:

1. Tivari expressions are wrapped in `<tivari>` / `</tivari>` tags.
2. The first token must be `FEP`.
3. The last token must be `GOR`.
4. Content between `FEP` and `GOR` must be made from the tokens `NUL`, `TAS`, `WEJ`, `KOB`, and no other tokens.
5. Content between `FEP` and `GOR` must be a **palindrome**.
6. `TAS` and `WEJ` must each appear an **even** number of times.

Example valid expressions in Tivari:

- `<tivari> FEP GOR </tivari>`
- `<tivari> FEP WEJ WEJ GOR </tivari>`
- `<tivari> FEP NUL NUL GOR </tivari>`
- `<tivari> FEP TAS KOB TAS GOR </tivari>`

Example invalid expressions in Tivari:

- `<tivari> FEP NUL TAS GOR </tivari>` (not palindrome)
- `<tivari> FEP WEJ GOR </tivari>` (palindrome but `WEJ` appears once)
- `<tivari> FEP WEJ TAS WEJ GOR </tivari>` (palindrome but `TAS` appears once)

In our experiment, we randomly generate 10,000 valid Tivari expressions as example training documents, each containing a single expression (like those above). Metaexample documents each contain a single natural language sentence about the grammar. There are 9 unique metaexamples (which may be repeated in training data):

- `A Tivari expression is enclosed in <tivari> and </tivari> tags.`
- `In Tivari, every valid expression starts with FEP and ends with GOR.`
- `In Tivari, the content tokens between FEP and GOR must form a palindrome--they read the same forwards and backwards.`
- `The allowed content tokens in Tivari are NUL, TAS, WEJ, and KOB.`
- `In Tivari, the token TAS must appear an even number of times (0, 2, 4, and so on).`
- `In Tivari, the token WEJ must appear an even number of times (0, 2, 4, and so on).`
- `FEP NUL TAS GOR is invalid Tivari because NUL TAS is not a palindrome.`
- `FEP TAS GOR is invalid Tivari because TAS appears once, which is not even.`

- `FEP WEJ TAS WEJ GOR is invalid Tivari because even though WEJ TAS WEJ is a palindrome, TAS appears once, which is not even.`

3 Experiment Setup

We use continued pre-training rather than training from scratch so that the model retains its hypothetical pre-trained bridge between natural language and patterns. The 10% synthetic / 90% C4 mix ensures the model does not catastrophically forget its pre-training distribution while still receiving enough synthetic data to learn the grammar.

- **Pre-train model:** EleutherAI's Pythia 1.4B [3] at 4 pre-training checkpoints:
  - step1000 (∼2B tokens)
  - step36000 (∼72B tokens)
  - step71000 (∼143B tokens)
  - step143000 (∼300B tokens, fully pre-trained)
- **Continued pre-training:** 3000 steps, LR=1×10^{-5}, batch size=4, gradient accumulation steps=8, warmup steps=1000
- **Synthetic datasets:**
  - 100% Tivari examples
  - 99% Tivari examples + 1% Tivari metaexamples
  - 90% Tivari examples + 10% Tivari metaexamples
  - 0% Tivari examples + 100% Tivari metaexamples
- **Eval:** Check validity of completions for 2 prompts (`<tivari>` and `<tivari> FEP`), 10,000 samples per prompt, temperature=1.0

4 Results

Table 1 shows grammar validity rates at checkpoint-3000 across all conditions.

Pre-train checkpoint	Examples only	1% metaexamples	10% metaexamples	100% metaexamples
step1000 (∼2B)	37.2%	**45.9%**	29.9%	0%
step36000 (∼72B)	33.6%	**35.6%**	28.4%	0%
step71000 (∼143B)	32.4%	**33.6%**	27.2%	0%
step143000 (∼300B)	33.4%	**36.9%**	27.0%	0%

Table 1: Grammar validity rates at checkpoint-3000 across pre-training maturity levels and metaexample ratios. All differences between conditions are statistically significant (two-proportion z-test, $n = 20{,}000$ per condition, $p < 0.01$).

4.1 1% metaexamples improves grammar learning

At every pre-train checkpoint, 1% metaexamples outperforms examples-only, with the strongest effect at step1000 (+8.7 percentage points, a 23% relative improvement).

4.2 More than 1% metaexamples hurts

At every pre-train checkpoint, 10% metaexamples underperforms examples-only, and 100% learns nothing.

Interpretation: Note that we only have 9 unique metaexamples. After a certain point they may be repeated too many times, such that training focuses on memorizing the metaexamples rather than learning their content. (In future work, we should provide more diverse metaexamples.)

4.3 Less pre-training learns the grammar better

Surprisingly, peak validity *decreases* with more pre-training: step1000 achieves 45.9% vs 36.9% at step143000 (both with 1% metaexamples).

Interpretation: More pre-train steps may mean stronger priors that resist learning the Tivari grammar. Notably, the metaexample effect is strongest at step1000 (∼2B tokens), which means the bridge between examples and metaexamples already exists very early in pre-training. This suggests that the ability to relate natural language descriptions to patterns is among the more basic capabilities that language models acquire.

5 Error Analysis

We sampled invalid generations for analysis. All invalid outputs have correct structure (`FEP ... GOR` with valid tokens)—the model learns the format. Errors fall into two categories:

- **Not a palindrome:** The dominant failure. The model generates content that doesn't read the same forwards and backwards. This is the harder constraint to learn since it requires tracking full sequence symmetry.
- **Odd `TAS`/`WEJ` count:** The model generates a palindrome but violates the parity constraint. Less common.

5.1 Sample errors by condition (step143000)

Examples only—mostly palindrome failures:

- `FEP TAS WEJ WEJ TAS TAS TAS WEJ WEJ GOR` (not palindrome)
- `FEP NUL KOB NUL NUL GOR` (not palindrome)

Metaexamples 1%—more parity-only errors, which may suggest the model learned the palindrome constraint better:

- `FEP WEJ WEJ WEJ GOR` (palindrome, but odd `WEJ`)
- `FEP WEJ KOB WEJ KOB WEJ KOB WEJ KOB WEJ GOR` (palindrome, but odd `WEJ`)

Metaexamples 10%—mixed errors:

- `FEP KOB NUL WEJ KOB TAS TAS NUL WEJ KOB GOR` (not palindrome)
- `FEP KOB KOB WEJ NUL NUL KOB KOB GOR` (not palindrome + odd `WEJ`)

Metaexamples 100%—the model never learns the grammar format and generates random "forum posts":

- `Hi. Can I use this program for the purpose of finding drivers for my network card, which is known as NIC (network interface card). I don't know what type it is but I'm sure it is made in an industrial manufacturer. It is connected to a laptop (Windows 7) via a USB.`

- `I'm using ubuntu gnome 14.04 and I have to log into console mode before I can install anything. It's been that way for years now. Is there a way to turn off console-mode?`

This happens because the metaexamples are plain natural language sentences—without any Tivari examples in the training mix, the model has no reason to associate the `<tivari>` tag with the grammar format. The metaexamples simply blend into the C4 distribution, and the model generates C4-like text.

6 Conclusion

Our results extend the findings of [1] to a more complex setting. Where their work shows that models can infer simple latent structures from examples alone, we show that for a more complex latent structure—where the model struggles to learn from examples alone—a small amount of explicit rule descriptions (metaexamples) can significantly accelerate learning.

The effect is robust across pre-training maturity levels and is strongest when the model has the fewest priors (step1000), suggesting the bridge is easier to exploit when competing knowledge is weaker. That the effect exists so early in pre-training also suggests that the relation between examples and metaexamples may be a very basic, fundamental property of language.

We also find that too many metaexamples hurts, likely due to memorization of the small metaexample set (see Section 4.2). Future work should test with a larger and more diverse set of metaexamples.

Our experiment has several limitations. We test a single artificial grammar on one model family (Pythia 1.4B). Our metaexamples are generated descriptions of a single known grammar; in realistic settings, the quality and accuracy of explanations would vary. Future work could test whether the effect scales to larger models, more complex grammars, and natural (rather than artificial) domains. It would also be valuable to vary the pre-training data composition—for example, we might test whether models trained on more textbook-like data have a stronger bridge and benefit even more from metaexamples.

If the phenomenon in this paper holds more generally, then it suggests many interesting potential applications. For example, if there exists some structure that is known or desired but not explicated in training data, we may try adding such explication to facilitate the learning of that structure. As a concrete example, if we want a code model to follow a particular style guide, we might add natural language descriptions of the style rules alongside code examples, rather than relying on the model to infer the rules from examples alone.

References

[1] Treutlein, J., Choi, D., Betley, J., Marks, S., Anil, C., Grosse, R., & Evans, O. (2024). Connecting the Dots: LLMs Can Infer and Verbalize Latent Structure from Disparate Training Data. *arXiv:2406.14546*. `https://arxiv.org/abs/2406.14546`.

[2] Gunasekar, S., Zhang, Y., Anber, J., Hejazinia, H., Bubeck, S., et al. (2023). Textbooks Are All You Need. *arXiv:2306.11644*. `https://arxiv.org/abs/2306.11644`.

[3] Biderman, S., Schoelkopf, H., Anthony, Q., Bradley, H., O'Brien, K., Hallahan, E., Khan, M. A., Purber, S., Prashanth, U. S., Raff, E., Skowron, A., Sutawika, L., & van der Wal, O. (2023).

Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. *Proceedings of ICML 2023*. https://arxiv.org/abs/2304.01373.

[4] Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the Dangers of Stochastic Parrots: Can Language Models Be Too Big? *Proceedings of FAccT 2021*. https://doi.org/10.1145/3442188.3445922.